

Introduction to Algorithm Engineering

Counting Triangles

Spring 2026
Lecture 7

Basics

- Let $G = (V, E)$ be an undirected graph with $n := |V|$, $m := |E|$.
- The problem we are interested in is to count the number of tuples of vertices x, y, z in G such that all of xy , yz , and zx are in E .
- Each such tuple corresponds to a K_3 or a triangle in G .

Basics

- A naïve algorithm is as follows.
- Run a 3-way nested loop to check if each tuple (x, y, z) of vertices form a triangle.
- Run time = $O(n^3)$.

Basics

- Two other ways:

1. Edge iteration based:

- a) For each edge uv in E , let $T(u, v)$ be the number of triangles with uv as one edge.
- b) Note that $T(u, v) = |N(u) \cap N(v)|$, where $N(v)$ is the neighbors of vertex v .
- c) Now, the number of triangles in $G = \sum_{uv \in E} T(u, v)$.
- d) Make sure you avoid counting duplicates.

Basics

- Two other ways

2. Vertex based counting

- a) Let v be a vertex in G and denote by $T(v)$ the number of triangles that v is a part of.
- b) Observe that $T(v) = |\{ (u, w) \mid u, w \in N(v) \text{ and } uw \in E \}|$.
- c) Now, the number of triangles in $G = \sum_{v \in V} T(v)$.
- d) Again, remove duplicates.

Edge Based Approach

- A popular approach to count triangles is to use the edge based approach.
- For each edge uv , $T(u,v)$ can be counted by finding the size of the intersection of $N(u)$ and $N(v)$.
- These algorithms are called also as intersection-based algorithms.
- Each intersection can be computed in time $O(\Delta(G))$ where $\Delta(G)$ is the degree of the graph.
- So, the total run time is $O(m.\Delta(G))$.

A Faster Approach

- Consider trying to minimize the number of intersections one has to do in the edge based approach.
- This fits well in the algorithm engineering perspective.
- Let us see how to find such a scheme.

Cover Edge Based Approach

- The notion of covers in a graph is well established.
- A vertex cover covers the edges.
- In that spirit, let us say that any edge e of a triangle in the graph G , is referred to as a cover-edge of the triangle.
- A set of edges C as a cover set for triangles if set $C \subseteq E$ is called a cover-edge set if it contains at least one cover-edge for every triangle in G .
- The term $c := |C|/|E|$ is called as the covering ratio.

Cover Edge Based Approach

- Notice that if C is a cover set for triangles, then only the end points of edges in C need to be checked for intersection of their neighborhoods.
- A low covering ratio c will mean that the work of the algorithm reduces.
- Our goal is to therefore find a cover set of triangles efficiently that is small enough.
 - The entire edge set E is a cover set, but has a covering ratio of 1! Not helpful.

Cover Edge Based Approach

- Consider the BFS tree of G .
- Classify the edges of G as
 - Tree edges: These edges belong to the BFS tree
 - Strut Edges: These are non-tree edges with endpoints on two adjacent levels in the BFS traversal
 - Horizontal Edges: These are non-tree edges with endpoints on the same level in the BFS traversal.

Claims

- Claim 1. Each triangle $\{u, v, w\}$ in a graph contains at least one horizontal-edge in an arbitrarily rooted BFS tree.
- Proof: A triangle is a path of length three that starts and ends at the same vertex.
- Suppose there are no horizontal-edges in the triangle. In that case, every edge in the path (i.e., a tree-edge or strut-edge) either increases or decreases the level by one.
- Since the path must end on the same level as the starting vertex, the number of edges in the path that decrease the level must be equal to the number of edges that increase the level.
- Consequently, such a path must have an even length.
- Therefore, we conclude that there must be at least one horizontal-edge in every triangle.

Claims

- Claim 2: All horizontal-edges in an arbitrarily rooted BFS tree form a valid cover-edge set.
- Proof: For any triangle in graph G , we can always find at least one horizontal-edge that serves as a cover-edge for that triangle.
- Thus, the set of all horizontal-edges constitutes a cover-edge set.

Claims

- Claim 3: Each triangle $\{u, v, w\}$ must contain either one or three horizontal-edges.
- The path corresponding to the triangle's three edges consists of an even number of tree-edges and strut-edges.
- So, there can be either 0 or 2 tree- or strut-edges within each triangle.
- In the case where there are 0 tree- or strut-edges,
 - all three edges of the triangle must be horizontal edges.
 - This is because the absence of tree- or strut-edges implies that the entire path is composed of horizontal-edges.
- In the case where there are 2 tree- or strut-edges,
 - The triangle contains exactly one horizontal-edge.
 - This is because having two tree- or strut-edges in the path means that there is one horizontal-edge connecting the remaining two vertices.

Pseudocode

Algorithm TriangleCount(G)

```
1:  $T \leftarrow 0$ 
2:  $\forall v \in V$ 
3:   if  $v$  unvisited, then BFS( $G, v$ )
4:  $\forall (u, v) \in E$ 
5:   if  $(L(u) \equiv L(v)) \wedge (u < v)$ 
6:      $\forall w \in N(u) \cap N(v)$ 
7:       if  $(L(u) \neq L(w)) \vee ((L(u) \equiv L(w)) \wedge (v < w))$  then
8:          $T \leftarrow T + 1$ 
9: return  $T$ 
```